

The Penetration Testing Lifecycle

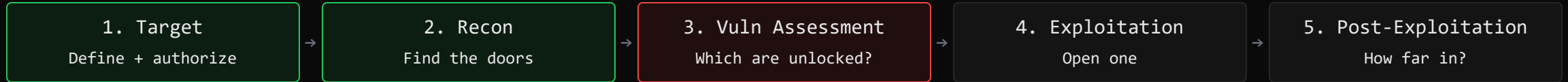
Day 3 – Vulnerability Assessment: Which Doors Are Unlocked?

Cybersecurity Academy • 5-Day Hands-On Program

Cybersecurity Academy • The Penetration Testing Lifecycle • Day 3

Where We Are

Yesterday we mapped every door. Today we check the locks – **without opening any.**



REMEMBER

Every phase's output is the **next phase's input**. Yesterday's list of open ports becomes today's list of suspects. Today's ranked list becomes tomorrow's target.

Learning Journey Check

Yesterday... We ran Nmap and got a map – open ports, services, version numbers.

Today... We take each version number and ask: *does it have a known weakness, and how bad is it?*

Tomorrow... We take the #1 entry on our ranked list and actually break in.

PRO TIP

The skill today is **not** "run a scanner". Anyone can run a scanner. The skill is turning 40 lines of scanner output into a 3-line prioritised list your client can act on.

What Is Vulnerability Assessment?

Vulnerability assessment is **finding and ranking** the weak points in the attack surface.

It IS

- Matching services + versions to **known** vulnerabilities
- Reading severity (CVSS) and exposure
- Checking whether a public exploit exists
- Producing a **prioritised** findings list

It is NOT

- Logging in
- Running an exploit
- Changing anything on the target
- "We found 14 issues" with no ranking

DEFINITION

The moment you would *use* a weakness instead of *documenting* it, you have left this phase. That is Day 4.

Where It Sits

	Vulnerability Scan	Vulnerability Assessment	Penetration Test
Tool runs	Yes	Yes	Yes
Human judgement	Little	A lot – validate + rank	A lot
Exploitation	No	No	Yes
Output	Raw findings list	Prioritised findings + fix order	Report with proven impact

REMEMBER

A scan is a step *inside* an assessment. The assessment is the scan **plus** the thinking: is this real? how exposed? how urgent?

What Is a Vulnerability?

A known weakness in a specific version of software.

DEFINITION

Stress two words. **Known** – someone has published it, given it a CVE number. **Version** – it affects `vsftpd 2.3.4`, not "FTP in general".

Where they come from – five buckets:

Bucket	Example
Outdated version	<code>Apache 2.2.8</code> from 2008, unpatched since
Misconfiguration	Directory listing on, default page left up
Default / weak credentials	<code>root</code> / no password on MySQL
Missing patch	Vendor fixed it; this box never updated
Needless exposure	Database port open to the whole network

The Manual Chain – Learn This Before Any Scanner

One Nmap line becomes a decision, in five steps:

```
nmap line      "22/tcp open ssh OpenSSH 4.7p1"
  | search ▼
NVD / Exploit-DB "OpenSSH 4.7p1 exploit"  · "... CVE"
  | read ▼
CVE + CVSS      CVE-XXXX-YYYY    score 7.5 (High)
  | check ▼
public exploit? searchsploit / Metasploit module?  yes / no
  | ▼
PRIORITY        where does this sit on the ranked list?
```

REMEMBER

A scanner does all five in one second. You learn them by hand so you can tell when it's **wrong**.

Reading a CVE: the CVSS Score

Every published vulnerability gets a **CVSS** score – severity from **0.0** to **10.0**.

0.1–3.9

LOW

4.0–6.9

MEDIUM

7.0–8.9

HIGH

9.0–10.0

CRITICAL

AV:N

remote – over the network

PR:N

no credentials needed

PRO TIP

You don't calculate CVSS – you read it. The two fields that change your night: is it **remote** (no foothold needed) and is it **unauthenticated** (no login needed)?

The Scanner – What It Actually Is

What people think

- A hacking tool
- It "finds vulnerabilities"
- If it's quiet, we're secure
- Its output is the answer

What it is

- A **librarian with a checklist**
- It reads version banners and looks them up in a big list
- Fast. Thorough. **Not smart.**
- Its output is a **list of leads**

WARNING

The scanner says *"this version is associated with CVE-X."* It has connected two strings. It has proven **nothing** about this box.

Scanner Tools You'll Meet

Tool	Scope	Use today
<code>nmap --script vuln</code>	Network services	One step past yesterday's <code>-sV</code>
<code>nikto -h <url></code>	Web server	Outdated software, dangerous methods, config
<code>searchsploit <product ver></code>	Local Exploit-DB copy	"Is there public exploit code?"
Nuclei / OpenVAS (Greenbone)	Everything, at scale	The enterprise scanners – named, demo only

DEFINITION

Nmap + Nikto + a version search is a complete *manual* assessment for a small target. Nuclei / OpenVAS is the same idea with thousands of checks and a web dashboard.

False Positives – the Discipline

A scanner result is a **lead**, not a **finding**.

Why it lies

- **Backported patches** keep the old version number – Debian fixes the bug, banner still says **2.2.8**
- The CVE needs a **config** that isn't set here
- The service is behind something the scanner can't see

What you do

- Read the CVE's actual conditions
- Check the config / behaviour, don't trust the banner
- Mark it **confirmed**, **needs-check**, or **false positive**

PRO TIP

A smoke alarm going off doesn't mean the house is burning. It means **go and look**.

Our Northwind Target – The Reminder

From Day 2, in scope

- 10.10.14.5 – web app
- 10.10.14.6 – internal server
- Window: 18:00–22:00, Mon–Fri
- Everything else: **OUT**

Journal §3 – what lands on the page

Field	Today's value
Phase	3 – Vuln Assessment
Input	Day 2 port/version list
Action	Match → validate → rank
Output	Prioritised findings table

WARNING

Scanners are **louder** than recon – more probes, more log noise. Same rule as Day 1: **only** the two authorised IPs.

From Yesterday's Map → Today's Suspect List

Day 2 gave us this. Now we interrogate every line.

10.10.14.5	WEB	10.10.14.6	SERVER		
22	ssh	OpenSSH 8.2p1	22	ssh	OpenSSH 8.2p1
80	http	nginx 1.18.0	445	smb	Samba 4.13.17
443	https	nginx 1.18.0			
3306	mysql	MySQL 8.0.28	◀ exposed on the network		

REMEMBER

Every version string is a question: *does this exact version have a rap sheet?* The exposed **3306** is a question about **exposure** even before we find a CVE.

Triage – This Is the Job

We don't hand the client 14 findings. We hand them a **ranked list**. Rank each finding by four things:

Factor	Question
Severity	What's the CVSS band – Critical / High / Medium / Low?
Exposure	Internet-facing? On the network? Behind a firewall?
Public exploit	Is there working code – <code>searchsploit</code> hit, Metasploit module?
Asset value	What's on that box – the customer database, or a test page?

DEFINITION

A **Medium** that is internet-facing with a Metasploit module often outranks a **Critical** that is firewalled with no public exploit. Count is not priority.

Triage – Worked Example (Northwind)

#	Finding	Sev	Exposed?	Exploit?	Rank & why
A	MySQL 8.0.28 reachable on 3306	Med	Yes – network	config-dependent	1 – exposure + it's the data
B	nginx 1.18.0 – dated, minor CVEs	Low	Yes (80/443)	no	3
C	OpenSSH 8.2p1 – one High CVE	High	Yes (22)	PoC only	2 – high sev, watch it
D	Samba 4.13.17 on server 445	Med	network (internal)	version-specific	4 – check exact patch level

REMEMBER

#1 becomes tomorrow's exploitation target. The exposed database wins – not because of a CVE, but because of **where it is** and **what it holds**.

Attack and Defend – the Same List

Attacker reads it top-down

- Start at #1 – best chance, least effort
- Public exploit? Even better
- One confirmed hole is enough

Defender reads it top-down

- Patch / close #1 first
- Same severity + exposure maths
- This is a **vulnerability management program**

DEFEND

The output of a pentest's vuln-assessment phase is a gift to the defender: a ranked, justified to-do list. The attacker had to build it. You get to keep it.

Journal §3 – Write the Findings

Same habit. Every command, every result, every judgement call.

```
[2026-09-02 18:10] Phase 3 – Vulnerability Assessment
Input  : Journal §2 port/version list (both targets)
Action : searchsploit + nmap --script vuln on 10.10.14.5
Command: searchsploit "MySQL 8.0"      → no unauth RCE
Command: nmap --script vuln 10.10.14.5 → flags nginx 1.18 (minor), mysql exposed
Finding: 3306 exposed to network – no auth bypass found, but attack surface
Rank   : #1 (exposure + asset value) – candidate for Phase 4
Note   : validate MySQL user perms before calling it exploitable
```

REMEMBER

Timestamp • Input • Action • Command • Finding • Rank • Note. If the ranking isn't written with a reason, it didn't happen.

Live Demo (Follow Along)

Instructor drives; target is the lab box **10.89.1.10** (a deliberately old machine).

```
kali@kali:~$ nmap -sV 10.89.1.10          ◀ the door map (Day 2 recall)
21/tcp  open  ftp      vsftpd 2.3.4
22/tcp  open  ssh      OpenSSH 4.7p1
80/tcp  open  http     Apache httpd 2.2.8
139,445 open  smb      Samba 3.x
3306    open  mysql    MySQL 5.0.51a
kali@kali:~$ searchsploit vsftpd 2.3.4    ◀ public exploit? → YES
kali@kali:~$ nmap --script vuln -p21,80,139 10.89.1.10  ◀ the scanner's leads
kali@kali:~$ nikto -h http://10.89.1.10    ◀ web: Apache 2.2.8, old PHP
```

YOUR TURN

Run the **nmap -sV** and one **searchsploit** on the version of your choice. Copy the exploit name (or "none") into your Journal.

What the Demo Told Us

Findings on 10.89.1.10

- **vsftpd 2.3.4** – public backdoor exploit, unauth, remote
- **Samba 3.x** – usermap script, unauth RCE
- **distccd** – command execution, no auth
- **Apache 2.2.8** / old PHP – many web CVEs
- **MySQL 5.0.51a** – weak/blank credentials likely

Ranked

1. **vsftpd 2.3.4** – unauth + remote + public code
2. **Samba usermap** – unauth RCE, public code
3. **distccd** – command exec
4. MySQL creds – needs a check
5. Apache/PHP web CVEs – many, mostly need conditions

REMEMBER

Same four-move chain every time: **match** → **validate** → **check for exploit** → **rank**. The old box just makes the chain pay off on every single line.

In-Class Activity: Triage the Findings (15 min)

You get a sheet of 6 findings – service, CVSS score, exposure note, "public exploit? y/n".

Produce

- A **ranked** fix list (1 → 6)
- One sentence justifying your **#1**

Then, as a group

- Compare rankings – where do they differ?
- Did a Medium beat a Critical? Why?
- Would the ranking change if the client's crown-jewel DB moved?

Write your final ranked list into Journal §3.

What We Do NOT Do Today

NOT TODAY

No logging in. No running an exploit. No touching data. Confirmed weakness → write it down and rank it. Using it is Day 4.

NOT TODAY

No target that isn't on the list. Scanners send far more traffic than recon – a mis-aimed `nmap --script vuln` at a real host is loud *and* illegal.

DEFEND

On the blue side, a sudden burst of version probes and script traffic from one source is exactly what your IDS should alert on. Your scan should be visible in your own logs.

Recap – Day 3

You can now

- Define vulnerability assessment vs scan vs pentest
- Run the manual chain: version → CVE → CVSS → exploit?
- Read a CVSS band and the remote/unauth fields
- Spot a likely false positive
- **Triage** findings into a ranked, justified list

The mindset

- The scanner is a librarian, not an oracle
- A lead is not a finding – validate
- Count is not priority – rank by sev × exposure × exploit × value
- Same list serves attack and defence

REMEMBER

Tomorrow – **Exploitation**. We take entry #1 on your ranked list and prove it. Bring your Journal with §3 finished.

Homework

1. **CVSS** – write, in your own words: what score range is "Critical"? What does *attack vector: Network* mean, and why does *Privileges Required: None* make a bug worse?
2. **One CVE** – pick one from today's demo. One sentence on what it lets an attacker do. Does the attacker need valid credentials first – yes or no?
3. **Journal** – make sure §3 has your ranked findings table, each row with a one-line reason.
4. **Read** one page: what "vulnerability management" means as an *ongoing program* (scan cadence, patch SLAs) – not a one-time scan.

PRO TIP

Tomorrow moves fast. The better you can read a scan result and name the weak line, the sooner you're through the door.

Questions?

Think about:

1. A scanner flags 30 findings. Your client can fix 3 this month. How do you choose – and how do you defend the choice?
2. Why is "the scanner said it's vulnerable" not good enough to put in a report?
3. Recon and vuln assessment are kept as separate phases. What would go wrong if you merged them?

PRO TIP

Vuln assessment's defence: a patch process with a clock on it. A known CVE is only dangerous while it stays unpatched – the ranked list tells you which one to fix first.